

PROGRAMACIÓN DEL ROBOT DOBOT MG400



INTRODUCCIÓN

Este proyecto muestra cómo controlar el movimiento de un robot mediante programación en bloques y comunicación Modbus. A través de ejemplos prácticos, como trayectorias automáticas y procesos de paletizado, se demuestra cómo el robot puede ejecutar tareas con precisión y eficiencia.

La programación por bloques facilita la interacción con sensores y actuadores, permitiendo una operación autónoma y segura en entornos industriales.



DobotStudio Proユーザーマニュアル (MG400 & M1 Pro)

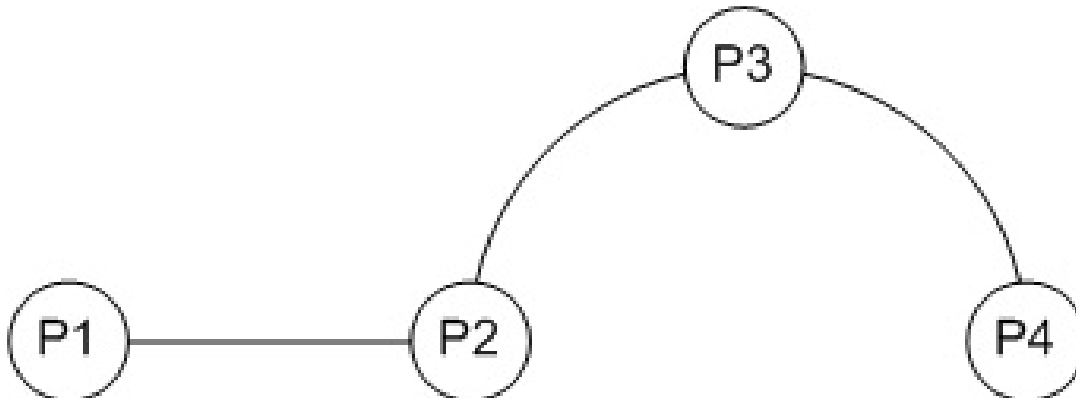


Shenzhen Yuejiang Technology CO., Ltd.

CONTROLAR EL MOVIMIENTO DEL ROBOT

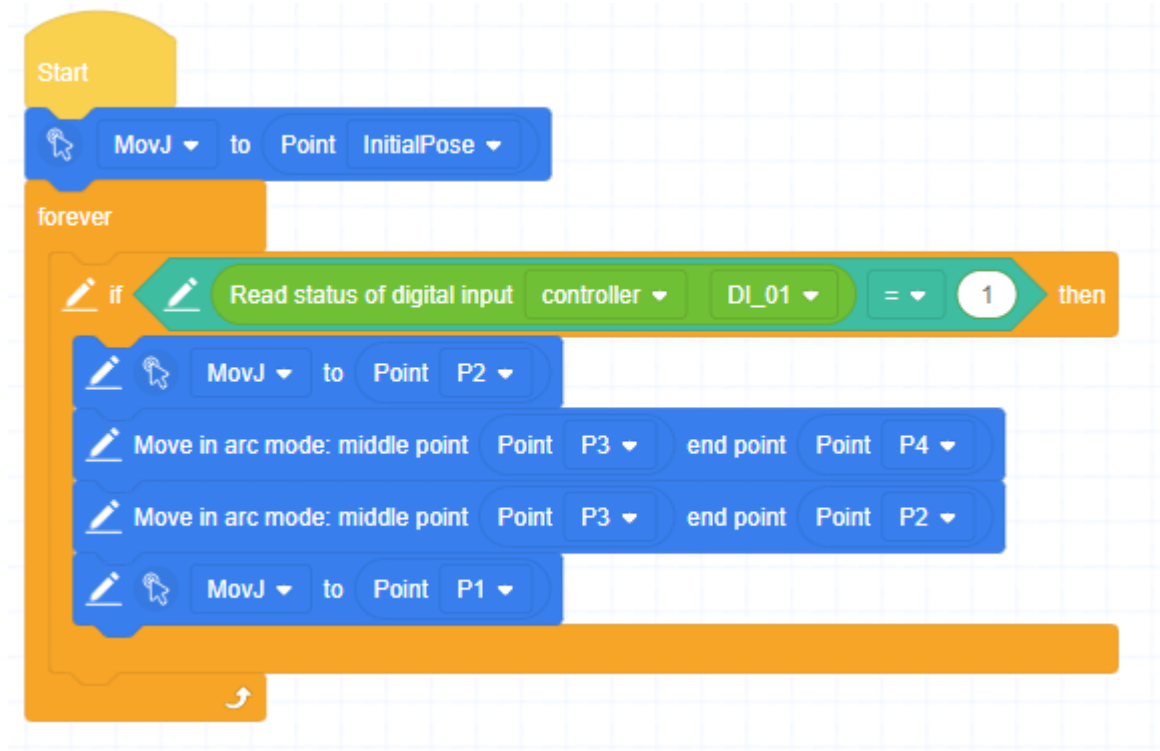
EJEMPLÓ 1 TIPOS DE MOVIMIENTOS

- Cuando el programa sea iniciado, el robot se moverá desde la posición inicial hasta la segunda mediante un desplazamiento lineal.
- Luego, continuará hacia el punto 3 y alcanzará el punto 4 utilizando un movimiento en modo *joins*.
- Al finalizar este trayecto, retornará siguiendo el mismo recorrido en sentido inverso. Si la entrada digital 1 no está activada, el robot permanecerá detenido.



EJEMPLO PROGRAMA EN BLOQUES

Movimientos del robot:



LECTURA Y ESCRITURA DE DATOS DE REGISTRO MODBUS

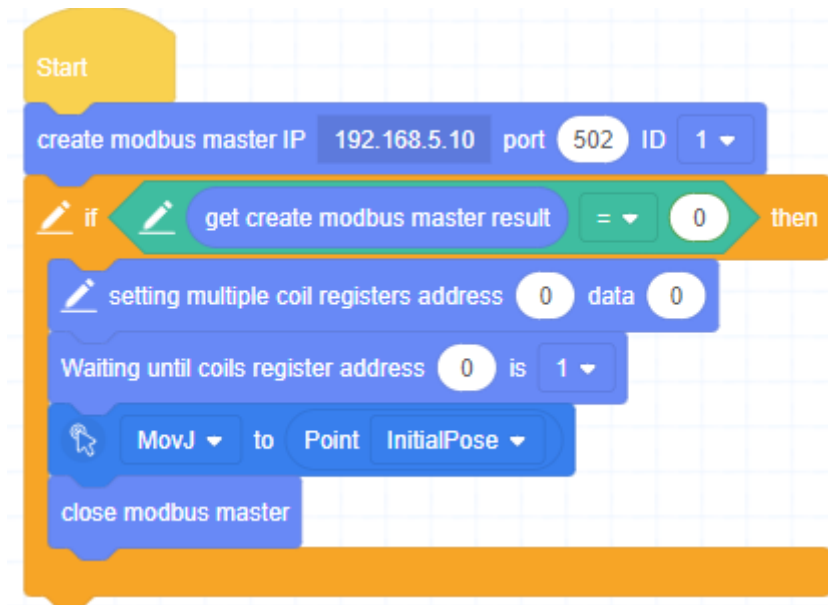
¿QUÉ ES EL MODBUS?

- **Modbus en un Dobot MG400** es un protocolo de comunicación que permite al robot interactuar con otros dispositivos industriales, como PLCs (controladores lógicos programables), sensores, o sistemas SCADA. Es una forma estandarizada y eficiente de enviar y recibir datos entre el Dobot y otros equipos, facilitando la integración en entornos de automatización industrial.

EJEMPLO

Un PLC puede enviar una señal al Dobot MG400 mediante Modbus para que inicie una secuencia de movimiento. Si el valor es 1, el robot se moverá a la posición P1.

- Programación en bloques:



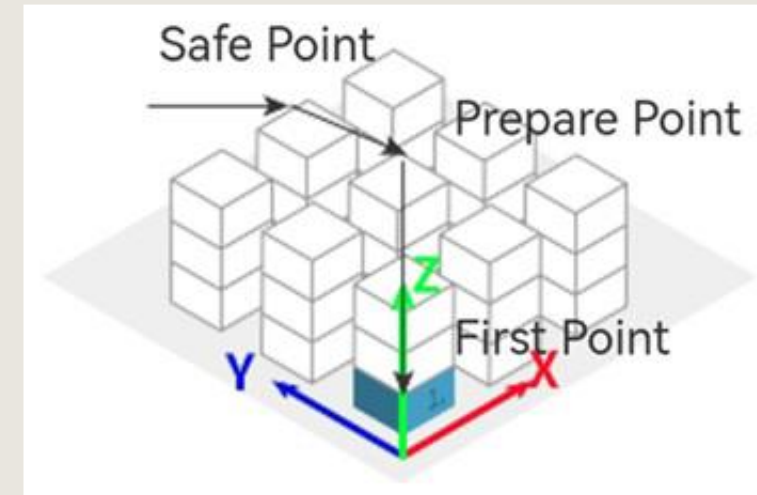
MODBUS PASO A PASO

- 1.Crear la estación maestra:** Configure la dirección IP apuntando hacia el dispositivo esclavo. Establezca el puerto y el ID utilizando los valores predeterminados. En esta demostración, la IP corresponde a la del robot, ya que se emplea su módulo esclavo para una verificación rápida.
- 2.Verificar la creación de la estación maestra:** Asegúrese de que la estación maestra se haya creado correctamente. Solo si esta operación tiene éxito se ejecutarán los pasos siguientes; de lo contrario, el programa se detendrá inmediatamente.
- 3.Restablecer el registro de bobina 0:** Si el valor del registro de bobina 0 del robot ha sido modificado, podría interferir con el funcionamiento del programa. Por lo tanto, primero es necesario establecer su valor en 0.
- 4.Esperar el cambio del valor del registro de bobina 0:** El programa debe esperar hasta que el valor del registro de bobina 0 cambie a 1.
- 5.Mover el robot:** Controle el robot para que se desplace al punto P1, el cual ha sido definido previamente por el usuario.
- 6.Cerrar la estación maestra:** Una vez completadas las operaciones, cierre la estación maestra.

PALETIZAR

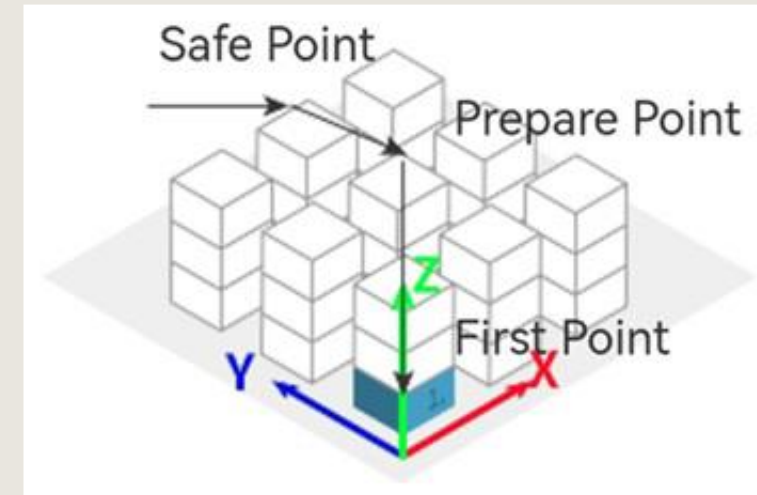
¿QUÉ ES EL PALETIZADO

- El proceso de paletizado permite resolver diversos problemas, especialmente en situaciones donde los materiales a transportar están dispuestos de forma regular y uniformemente espaciados. En estos casos, enseñar al robot cada posición de manera individual puede resultar ineficiente y propenso a errores



EJEMPLO DE PALETIZADO

- Si el objetivo es apilar el material en forma de un cubo, sería necesario definir manualmente cada una de las posiciones de la pila de destino, lo que puede ser una tarea tediosa y poco práctica. El uso de funciones de paletizado automatizado simplifica este proceso, mejora la precisión y aumenta considerablemente la eficiencia operativa.



EXPLICACIÓN PASO A PASO

Punto seguro (P1): Es una posición a la que el robot debe desplazarse durante las operaciones de ensamblaje o desmontaje de pilas, garantizando una transición segura. Generalmente, se define como un punto elevado sobre el punto de recogida, para evitar colisiones con el entorno o con otros objetos.

Punto de recogida (P2): No es necesario enseñar manualmente cada posición de preparación o destino para cada material. En su lugar, se recomienda utilizar la función de **Configuración del tipo de pila**, que permite definir automáticamente las posiciones en función del patrón de apilamiento.

Además, se asume que una herramienta de sujeción, como una **pinza** o una **ventosa**, ha sido instalada en el extremo del brazo robótico. Esta herramienta está controlada mediante la **salida digital DO1**, la cual permite activar o desactivar la acción de agarre para manipular los materiales.

CONFIGURACIÓN DEL BLOQUE

- Arrastre el **bloque de paletizado** al área de programación y haga clic sobre él para abrir el **panel de configuración de paletizado**.
- Según las dimensiones del patrón de apilamiento, será necesario determinar si se trata de un paletizado **unidimensional**, **bidimensional** o **tridimensional**:
- **Unidimensional**: Solo se utiliza una fila lineal de posiciones (por ejemplo, en una sola dirección).
- **Bidimensional**: Se organiza en filas y columnas dentro de un solo plano (X e Y).
- **Tridimensional**: Involucra múltiples capas en altura (X, Y y Z), formando un volumen completo como un cubo o paralelepípedo.
- Esta configuración define cómo se distribuirán los puntos de colocación dentro del bloque de paletizado.



Pallet panel

Pallet dimension: One-dimensional

Number of X direction of 1: 10

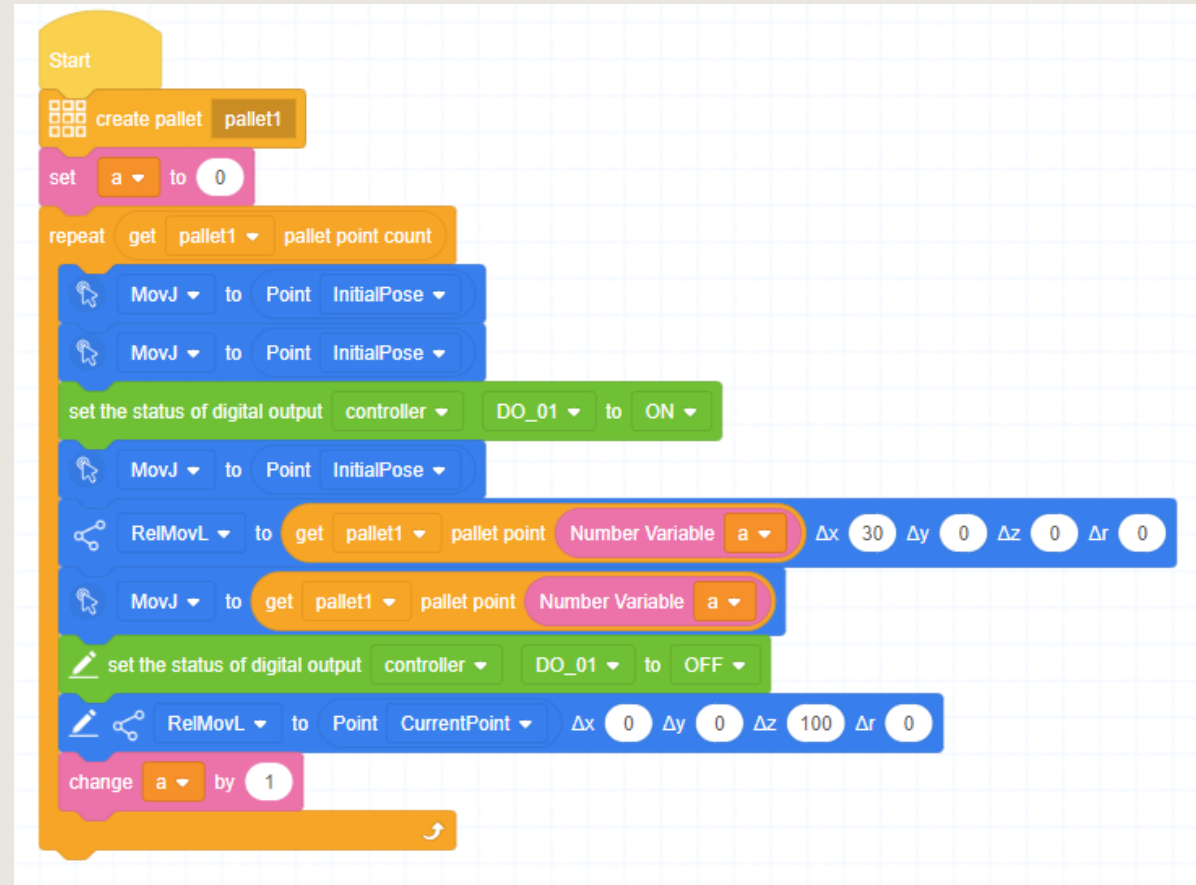
Point configuration: 1 2

Point1: Please select Custom

Cancel Save

EJEMPLO DE PROGRAMA TRIDIMENSIONAL

1. Crear pallet1.
2. Cree una variable numérica personalizada y configúrela en 1, que se utiliza para registrar los tiempos de repetición
3. Ejecute los comandos siguientes cíclicamente y establezca el número de veces en el número total de puntos correspondientes al palet.
4. El robot se desplaza sobre el punto de recogida (P1).
5. El robot se desplaza hasta el punto de picking (P2).
6. Ponga DO1 en ON para controlar la pinza y recoger el material.



Make a Variable

Type: ☒ Number ☐ String

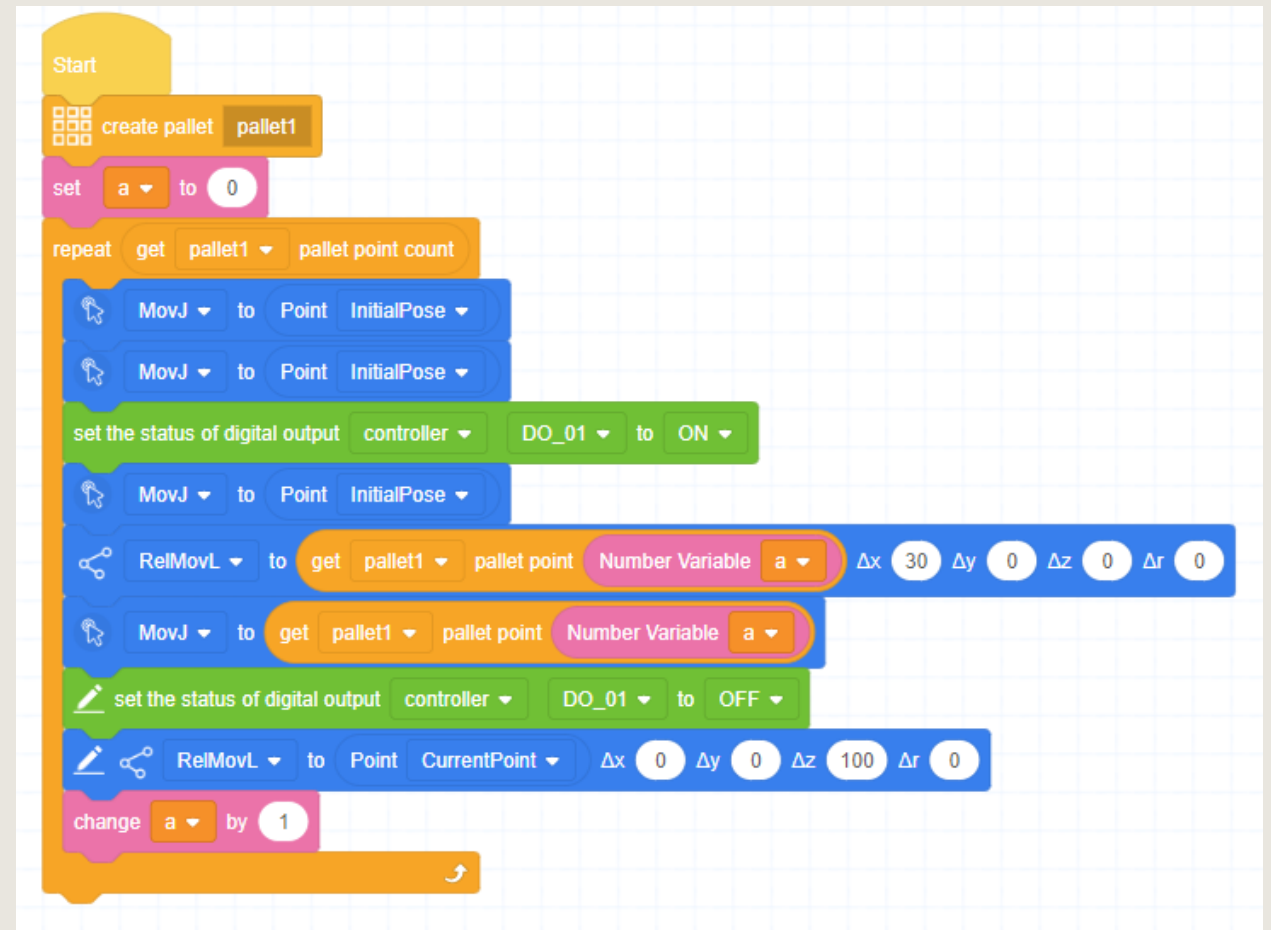
Name:

Cancel

Confirm

EJEMPLO DE PROGRAMA TRIDIMENSIONAL

7. El robot regresa sobre el punto de recogida (P1).
8. El robot se mueve a 100 mm sobre el punto de paleta actual.
9. El robot se desplaza al punto de palé actual.
10. Ajuste DO1 a OFF para controlar la pinza y liberar el material.
11. El robot regresa a 100 mm sobre el punto de paleta actual.
12. Los tiempos de repetición se incrementan en 1. Vuelva al paso 4.



Make a Variable

Type: ☒ Number ☐ String

Name:

Cancel

Confirm

DESCRIPCIÓN DE LOS BLOQUES

BLOQUES DE EVENTOS

En DobotStudio Pro, los **bloques de eventos** son fundamentales para estructurar y controlar la ejecución de programas en robots colaborativos de las series MG400 y M1. A continuación, se describen los principales bloques de eventos

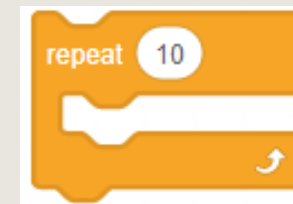
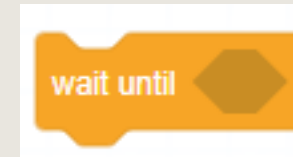
- **Comando de Inicio:** Este comando se usa **una sola vez por programa** y debe colocarse **al principio**. Su función es **indicar dónde empieza el programa principal**, es decir, desde qué punto el robot va a comenzar a ejecutar las instrucciones.
- **Comando de Inicio de Subproceso:** Este comando **inicia un subproceso**, que es como una tarea secundaria que se ejecuta **al mismo tiempo** que el programa principal. Pero **no puede controlar directamente al robot** (no mueve el brazo ni activa salidas). Se usa solo para hacer **cálculos, usar variables o lógica interna**, como operaciones matemáticas o verificar condiciones.



BLOQUES DE CONTROL

Estos bloques permiten gestionar el flujo de ejecución dentro de la programación del robot, facilitando tanto el control lineal como la lógica condicional y repetitiva.

- **Esperar hasta:** Este comando **pausa el programa** y solo continúa cuando **se cumple una condición** (por ejemplo, que una variable tenga cierto valor). Se usa para **esperar a que algo pase** antes de seguir, como un sensor activado o una señal específica.
- **Repetir n veces:** Este bloque sirve para **repetir varios comandos** una cantidad de veces que tú elijas. Todo lo que esté dentro del bloque se ejecutará **una y otra vez**, hasta completar el número de repeticiones indicado. Es ideal para **hacer ciclos con repeticiones fijas**.



BLOQUES DE CONTROL

- **Repetir continuamente:** Este bloque **repite las instrucciones una y otra vez**, sin parar.
El ciclo solo se detiene cuando el programa encuentra un **bloque de “Fin de la repetición”** dentro del mismo bucle.
Es útil cuando no sabes cuántas veces se debe repetir, pero sí sabes cuándo terminar.
- **Fin de la repetición:** Este bloque sirve para **salir de un ciclo antes de que termine**.
Cuando el programa lo ejecuta, **detiene el ciclo actual** (ya sea “Repetir n veces” o “Repetir continuamente”) y continúa con las instrucciones que están después del ciclo.
Se usa para **romper un bucle** si se cumple cierta condición.



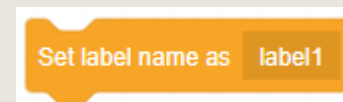
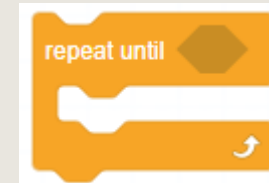
BLOQUES DE CONTROL

- **Si... Entonces:** Este bloque **evalúa una condición** (algo que puede ser verdadero o falso). **Si la condición es verdadera**, se ejecutan los bloques que están dentro. **Si es falsa, se salta** y continúa con lo que viene después. Es como decir: *“Si pasa esto, haz esto otro”*.
- **Si... entonces... Más:** También evalúa una condición, pero con **dos posibles caminos**: Si la condición **es verdadera**, se hacen las instrucciones del primer bloque. Si la condición **es falsa**, se hacen las instrucciones del bloque **“else”**. Es como decir: *“Si pasa esto, haz A; si no pasa, haz B.”*



BLOQUES DE CONTROL

- **Repetir hasta:** Este bloque repite las instrucciones que contiene **una y otra vez**, hasta que **se cumpla una condición**. Mientras la condición **sea falsa**, el ciclo continúa. Cuando la condición **se vuelve verdadera**, el ciclo termina y el programa **sigue con lo que viene después**. Es útil para repetir acciones **hasta que algo suceda**.
- **Establecer etiqueta:** Este bloque **crea una marca o punto específico** en el programa. Sirve para que, más adelante, el programa **pueda saltar directamente a ese punto** usando el bloque “Ir a etiqueta”. Es útil para **organizar o controlar el flujo** del programa con saltos intencionados.



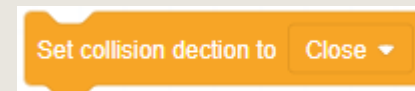
BLOQUES DE CONTROL

- **Ir a etiqueta:** Este bloque hace que el programa **salte directamente** a una parte específica del código marcada con una **etiqueta**. Desde ahí, el programa **continúa ejecutando normalmente**. Es útil para **repetir acciones** o **organizar mejor el flujo** del programa.
- **Comandos de plegado:** Estos bloques sirven para **esconder o mostrar** los bloques que están dentro de otros. **No cambian** cómo funciona el programa. Solo sirven para que el código **se vea más limpio y ordenado**. Son útiles cuando el programa es muy largo y quieres enfocarte en partes específicas.



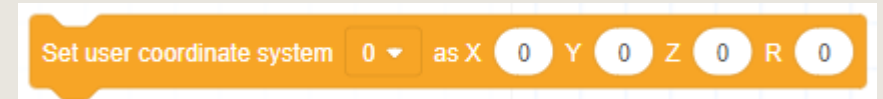
BLOQUES DE CONTROL

- **Pausa:** Este bloque **detiene el programa** justo en ese punto. Es útil cuando necesitas **esperar una acción manual, revisar algo** o simplemente hacer una pausa antes de seguir con la ejecución.
- **Establecer la detección de colisiones:** Este bloque permite **ajustar qué tan sensible es el robot** a posibles colisiones mientras se ejecuta un proyecto. Solo funciona **durante la ejecución**. Cuando el programa se detiene, la configuración **vuelve a la original**. Es útil para aumentar la seguridad o evitar daños en tareas delicadas.



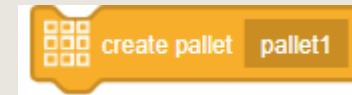
BLOQUES DE CONTROL

- **Modificar el sistema de coordenadas del usuario:** Este bloque permite **cambiar el punto de referencia** desde donde el robot mide sus movimientos, definido por el usuario. El cambio es **temporal**: solo funciona mientras el proyecto está en ejecución. Al terminar el programa, **se regresa al sistema de coordenadas anterior**. Se usa cuando necesitas que el robot trabaje **desde otro punto de origen** específico, sin cambiar la configuración principal.
- **Modificar el sistema de coordenadas de la herramienta:** Este bloque permite ajustar **la posición y orientación de la herramienta** que usa el robot (por ejemplo, una pinza o lápiz). El cambio también es **temporal** y solo aplica durante la ejecución. Cuando el proyecto termina, se **restablece la configuración anterior**. Es útil cuando cambias de herramienta o su forma de uso durante una tarea específica.



BLOQUES DE CONTROL

- **Crear paletizado:** Este bloque sirve para **configurar cómo el robot colocará objetos** en un patrón ordenado (como una cuadrícula o pila). Puedes definir **cuántas filas, columnas y niveles** tendrá. El robot usará esta información para colocar los objetos de forma precisa y organizada. Es ideal para tareas como **empaquetar, ordenar o apilar piezas**.
- **Obtener el recuento de puntos de paletizado:** Este bloque **te dice cuántos puntos** hay en total en el patrón de paletizado que configuraste. Cada punto representa un lugar donde el robot debe colocar un objeto. Sirve para **saber cuántas posiciones hay disponibles** o cuántos movimientos realizará el robot dentro del patrón.



BLOQUES DE CONTROL

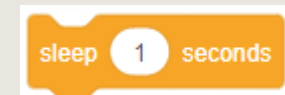
- **Obtener coordenadas de puntos de paletizado:** Esta función permite conseguir las coordenadas exactas de un punto dentro del paletizado, lo que ayuda a programar movimientos precisos del robot.
- **Establecer parámetros de carga:** Con esta opción se pueden ajustar los valores relacionados con el peso que puede cargar el brazo del robot, lo cual afecta la velocidad, la aceleración y la seguridad al moverse.

get pallet1 ▼ pallet point count

Set Payload Parametes: Payload g X-offset mm Y-Offset mm Servo Index(Optional)

BLOQUES DE CONTROL

- **Retrasar la ejecución:** Esta función detiene el programa por un tiempo que tú defines antes de continuar con las siguientes instrucciones. Es útil para sincronizar o esperar que termine otro proceso.
- **Movimiento en espera:** Se usa para hacer una pausa antes o después de un movimiento del robot. Esto ayuda a que el siguiente comando no empiece hasta que el movimiento anterior haya terminado, o para dejar un espacio entre movimientos.



BLOQUES DE CONTROL

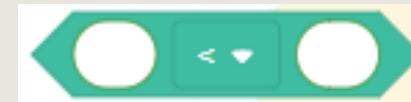
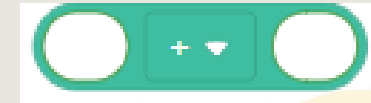
- **Obtener la hora del sistema:** Esta función consulta y muestra la hora actual del sistema. Es útil para llevar registros, sincronizar acciones o tomar decisiones basadas en el tiempo.

get system time

BLOQUES DE OPERADOR

Los bloques de Operador en Dobot Studio Pro, según el Apéndice B.2.3 del manual, permiten ejecutar operaciones matemáticas, lógicas y especiales dentro de tus programas. Incluyen aritméticos, comparación y lógica y especiales.

- **Comando aritmético:** Realiza operaciones matemáticas básicas como sumar, restar, multiplicar o dividir entre los números que le des.
- **Comando de comparación:** Compara dos o más valores para ver si son iguales, o si uno es mayor o menor que el otro, y da como resultado “verdadero” o “falso”.
- **Comando A y B:** Hace una operación lógica llamada AND entre dos valores (A y B). Devuelve “verdadero” solo si los dos valores son verdaderos; si no, devuelve “falso”.



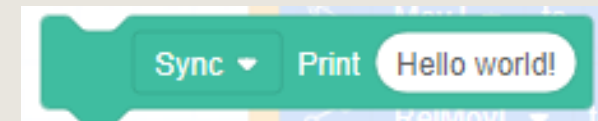
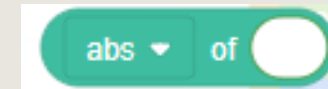
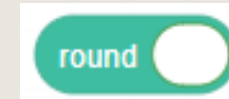
BLOQUES DE OPERADOR

- **Comando A o B:** Hace una operación lógica llamada OR entre dos valores (A y B). Devuelve “verdadero” si al menos uno de los dos valores es verdadero.
- **No es un comando:** No hace ninguna operación ni cambia nada. Se usa cuando no quieres que se realice ninguna acción.
- **Obtener resto:** Calcula y muestra el resto que queda después de dividir dos números.



BLOQUES DE OPERADOR

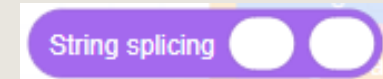
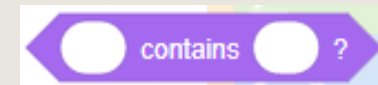
- **Operación de redondeo:** Redondea un número al entero más cercano o según la forma que se indique.
- **Operación monódica:** Realiza operaciones que necesitan solo un número, como obtener el valor absoluto (número sin signo), cambiar el signo, o aplicar funciones matemáticas a ese número.
- **Comando imprimir:** Muestra los datos o mensajes en la pantalla o registro para ayudar a revisar y controlar el programa mientras se ejecuta.



BLOQUES DE CADENA

Los bloques de Cadena permiten manipular texto en tu programa visual. Incluyen Concatenate para unir varias cadenas, Length, Substring , IndexOf, Trim y Replace.

- **Obtener carácter en una posición determinada de la cadena:** Devuelve la letra o símbolo que está en la posición que elijas dentro de un texto.
- **Determinar si la cadena A contiene la cadena B:** Revisa si un texto (cadena A) tiene dentro otro texto (cadena B) y dice “verdadero” si está o “falso” si no.
- **Conectar dos cadenas:** Une dos textos para formar uno solo, poniendo el segundo texto justo después del primero.



BLOQUES DE CADENA

- **Obtener la longitud de una cadena o matriz:** Esta función dice cuántos caracteres tiene un texto o cuántos elementos hay en una lista (matriz).
- **Comparar dos cadenas:** Compara dos textos letra por letra según su orden en el código ASCII, para saber cuál es mayor, cuál es menor o si son iguales.

String or array length

String comparison

BLOQUES DE CADENA

- **Convertir matriz a cadena:** Transforma una lista de elementos en un texto, uniendo cada elemento con un separador que elijas.
- **Convertir cadena a matriz:** Transforma un texto en una lista de elementos, separando el texto usando el separador que indiques.

Convert array to string Array:

☒

Separator:

☐

Convert string to array string:

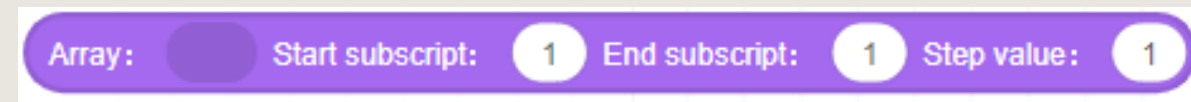
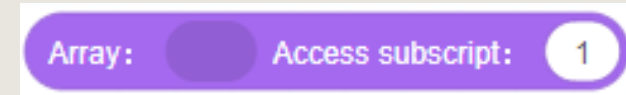
☐

Separator:

☐

BLOQUES DE CADENA

- **Obtener elemento en una posición determinada de la matriz:** Devuelve el elemento que está en la posición que indiques dentro de una lista.
- **Obtener múltiples caracteres especificados de una cadena:** Saca varios caracteres de un texto según las posiciones o rangos que señales.



BLOQUES DE CADENA

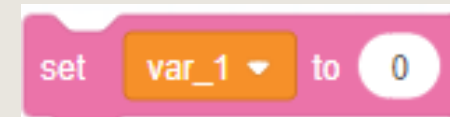
- **Establecer el carácter especificado en una matriz:** Cambia el valor del elemento que está en una posición específica dentro de una lista.

Set array elements Name: Index: Value:

BLOQUES DE PERSONALIZACIÓN

Los bloques de personalización nos permiten crear variables, arreglos, funciones y subrutinas.

- **Lllamar variable global:** Accede a las variables globales que están definidas en el programa para usarlas.
- **Establecer variables globales:** Asigna un valor a una variable global específica.



BLOQUES DE PERSONALIZACIÓN

- **Crear variables:** Haz clic para crear una nueva variable donde guardarás información.
- **Variable numérica personalizada :**
Se recomienda usar esta variable numérica personalizada que acabas de crear (su valor inicial es vacío o nulo) después de asignarle un valor.



Make a Variable

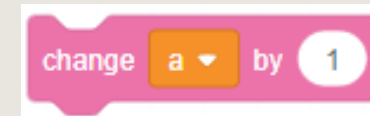
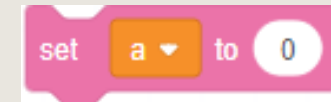


Number Variable

a ▼

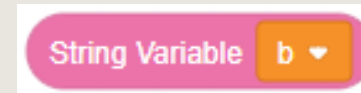
BLOQUES DE PERSONALIZACIÓN

- **Asignar valor a variable numérica personalizada:** Poner un número específico dentro de una variable para usarla después.
- **Incrementar valor de variable numérica:** Sumar un número a la variable que ya tiene un valor.



BLOQUES DE PERSONALIZACIÓN

- **Variable de texto personalizada:** Es una variable para guardar texto. Se recomienda usar la que acabas de crear (su valor inicial está vacío). También puedes cambiarle el nombre o eliminarla desde la lista de variables.
- **Asignar valor a variable de texto personalizada:** Poner un texto específico dentro de la variable para usarla después.



BLOQUES DE PERSONALIZACIÓN

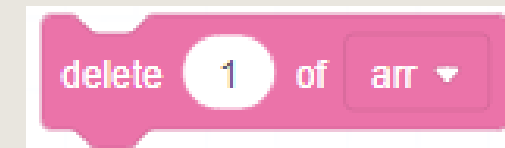
- **Crear arreglo:** Haz clic para crear una lista personalizada donde puedes guardar varios datos.
- **Arreglo personalizado:** La lista que acabas de crear está vacía al principio. Se recomienda usarla después de ponerle datos. Para cambiarle el nombre o eliminarla, haz clic derecho en PC o mantén pulsado en Android o iOS sobre el bloque en la lista.

Make a Array



BLOQUES DE PERSONALIZACIÓN

- **Agregar variable al arreglo:** Añade una variable al final de una lista específica.
- **Quitar elemento del arreglo:** Elimina un elemento de una lista que hayas seleccionado.



BLOQUES DE PERSONALIZACIÓN

- **¿Borrar todos los elementos del arreglo:**

Elimina todos los datos que están guardados dentro de una lista.

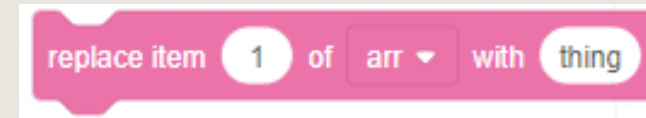


- **Insertar elemento en el arreglo:** Pone un dato en una posición específica dentro de la lista.



BLOQUES DE PERSONALIZACIÓN

- **Sustituir elementos del arreglo:** Cambia un dato en la lista por otro que tú elijas.
- **Obtener elementos del arreglo:** Toma el valor de un dato específico dentro de la lista.

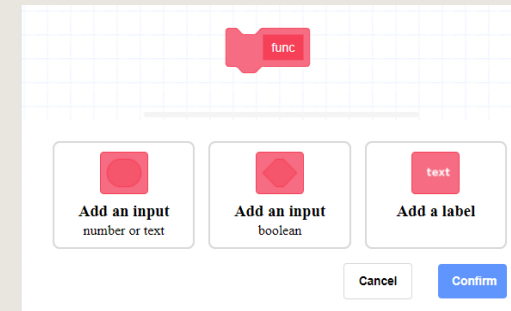


BLOQUES DE PERSONALIZACIÓN

- **Crear función:** Una función es un conjunto de instrucciones que hacen una tarea específica dentro del programa.
- **Definir función:** En esta parte, le pones un nombre a la función y decides cuántos datos (entradas) necesita para trabajar.

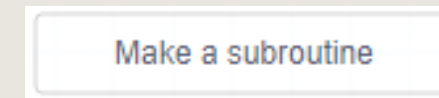
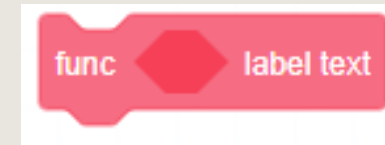


Make a Function



BLOQUES DE PERSONALIZACIÓN

- **Función personalizada:** Son bloques de código con un nombre y datos de entrada que tú defines. Se usan para ejecutar la función que creaste.
- **Crear subrutina:** Genera una pequeña parte del programa principal que puede usarse varias veces.
- **Subrutina en bloque y subrutina de script:** El bloque de subrutina sirve para llamar a esa parte del programa guardada. En PC, haciendo clic derecho, o en la app manteniendo pulsado, puedes editar o borrar la subrutina.



BLOQUES DE ENTRADAS Y SALIDAS

En Dobot Studio Pro, los bloques de Entradas y Salidas (I/O) permiten interactuar con el entorno físico del robot, gestionando señales digitales y analógicas para controlar dispositivos externos como sensores, interruptores, luces o actuadores.

- **Leer entrada digital:** Obtiene el estado (encendido o apagado) de una entrada digital específica.
- **Esperar entrada digital:** Detiene el programa hasta que la entrada digital cumpla una condición o hasta que pase un tiempo límite, y después continúa con el siguiente paso.

Read status of digital input

controller ▼

DI_01 ▼



wait digital input

controller ▼

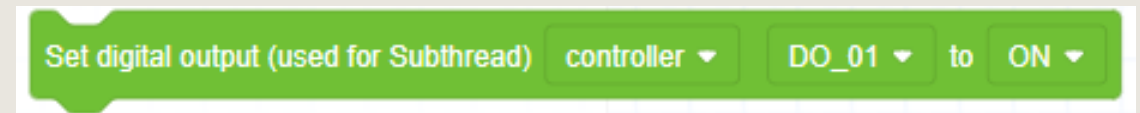
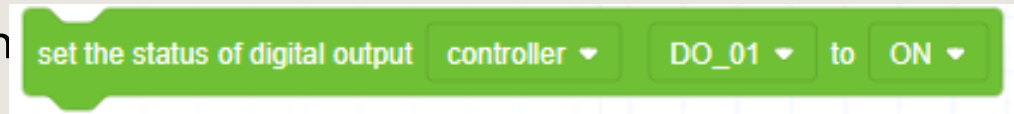
DI_01 ▼

ON ▼

0 S

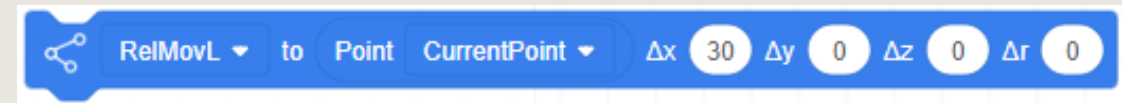
BLOQUES DE ENTRADAS Y SALIDAS

- **Configurar salida digital:** Enciende o apaga un puerto de salida digital.
- **Configurar salida digital (para subproceso):** Enciende o apaga un puerto de salida digital dentro de un subproceso (una parte separada del programa).
- **Configurar un conjunto de salidas digitales:** Define un grupo de salidas digitales que puedes controlar juntas. Solo arrastra el bloque y ajusta sus opciones.



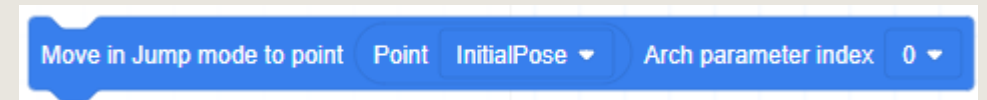
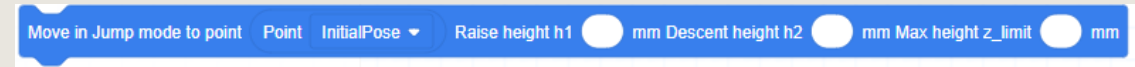
BLOQUES DE MOVIMIENTO

- **Desplazarse al punto objetivo:** Hace que el robot se mueva desde donde está hasta un punto específico. Después de poner el bloque en el programa, haz doble clic para ajustar opciones avanzadas.
- **Desplazarse al punto de destino (con desplazamiento):** Hace que el robot se mueva desde su posición actual hasta un punto específico, pero primero aplica un desplazamiento (un cambio de posición) antes de llegar al objetivo.



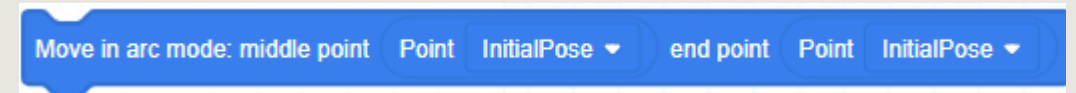
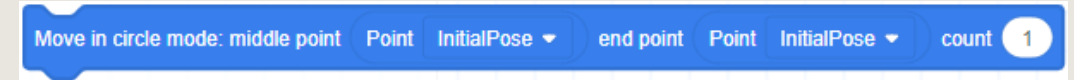
BLOQUES DE MOVIMIENTO

- **Movimiento de salto:** El robot se mueve rápido desde donde está hasta un punto destino dentro del sistema de coordenadas cartesianas, usando un modo llamado "puerta".
- **Movimiento de salto (con parámetros predefinidos):** El robot se mueve rápido desde su posición actual hasta el destino, usando ajustes de salto que ya están configurados.



BLOQUES DE MOVIMIENTO

- **Movimiento circular:** Hace que el brazo del robot se mueva en un círculo completo desde donde está, y luego regrese al mismo lugar después de dar una cierta cantidad de vueltas. Para que funcione, la posición inicial no debe estar en línea recta con los otros dos puntos del círculo.
- **Movimiento en arco:** Hace que el brazo del robot se mueva siguiendo una trayectoria curva (en forma de arco) desde su posición actual hasta un punto final. También necesita que la posición inicial no esté en línea recta con los otros dos puntos.



BLOQUES DE MOVIMIENTO

- **Gestionar el movimiento de la articulación auxiliar:** Controla el movimiento de una parte extra del robot (llamada articulación auxiliar). Solo se puede usar si ya se ha instalado y configurado esa parte.
- **Configurar la proporción de aceleración conjunta:** Ajusta qué tan rápido acelera la articulación del robot cuando se mueve.

Move in AuxJoint: motion angle / distance 20 speed percentage 50 acceleration percentage 50 Sync ▾

set joint acceleration percentage 10 %

BLOQUES DE MOVIMIENTO

- **Configurar la proporción de velocidad conjunta:** Ajusta qué tan rápido se mueven las articulaciones del robot durante un movimiento.
- **Configurar la proporción de aceleración lineal:** Controla qué tan rápido aumenta la velocidad cuando el robot se mueve en línea recta o en una trayectoria curva (en arco)

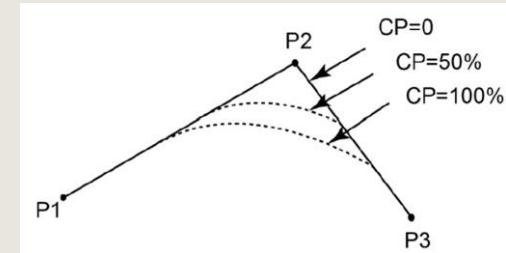
set joint speed percentage 10 %

set linear acceleration percentage 10 %

BLOQUES DE MOVIMIENTO

- **Configurar relación de velocidad lineal:** Define qué tan rápido se moverá el robot en línea recta. La velocidad real depende del porcentaje que pongas en este bloque, la velocidad configurada en la reproducción y la velocidad global.
- **Ajustar proporción de CP (punto continuo):** Controla qué tan suave será el movimiento del robot cuando pasa por varios puntos seguidos. El robot puede pasar por un punto intermedio haciendo una curva o girando en ángulo recto, sin detenerse.

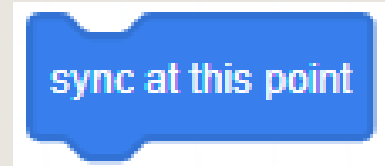
set linear speed percentage 10 %



set smooth transition percentage 10 %

BLOQUES DE MOVIMIENTO

- **Comando de sincronización:** Hace que el programa se detenga hasta que el brazo robótico termine todas las acciones anteriores. Luego, continúa con las siguientes instrucciones.



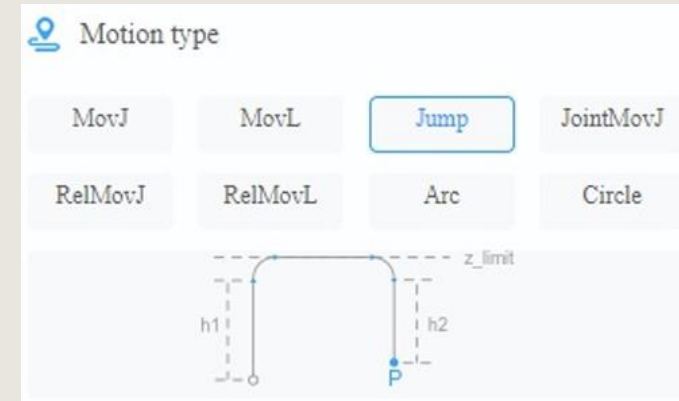
CONFIGURACIÓN AVANZADA DE MOVIMIENTO

- **MovJ:** Mueve el robot desde su posición actual hasta el punto de destino usando un movimiento en forma de arco. Este tipo de movimiento es más rápido y usa la rotación de las articulaciones del robot.
- **MovL:** Mueve el robot en línea recta desde su posición actual hasta el punto de destino. Es más preciso cuando necesitas que el extremo del brazo siga un camino recto.



CONFIGURACIÓN AVANZADA DE MOVIMIENTO

- **Saltar:** Mueve el robot desde donde está hasta un punto objetivo, levantándose primero para evitar obstáculos. Se usa el sistema de coordenadas cartesianas y un modo llamado “puerta”.
- **ConjuntoMovJ:** Hace que el robot se mueva desde su posición actual hasta un ángulo específico, usando el movimiento de sus articulaciones (como si giraran para llegar al objetivo).



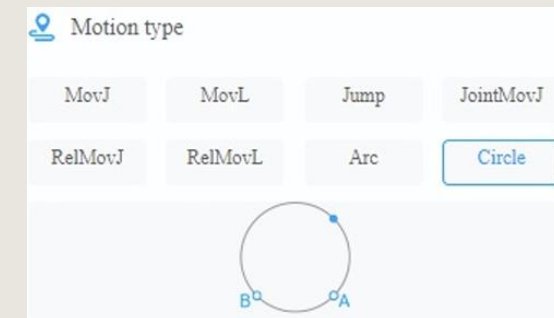
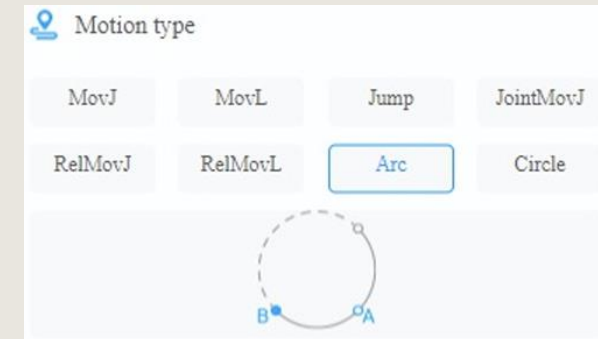
CONFIGURACIÓN AVANZADA DE MOVIMIENTO

- **RelMovJ:** Mueve el robot desde su posición actual hacia una nueva posición cercana, usando un movimiento suave de sus articulaciones. El punto de llegada se calcula como una posición relativa (es decir, a cierta distancia desde donde está), no como una posición fija.
- **RelMovL:** Mueve el robot desde donde está hacia una nueva posición cercana en línea recta, también usando una posición relativa en lugar de una fija..



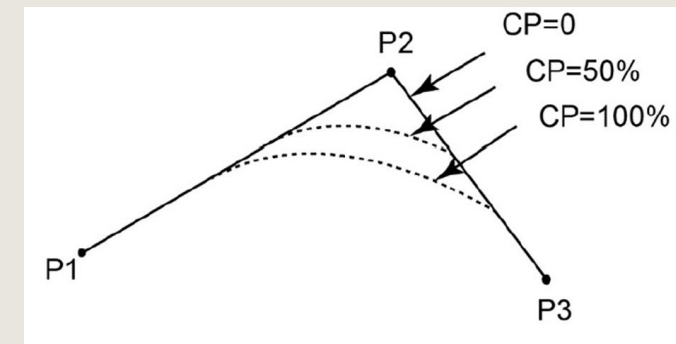
CONFIGURACIÓN AVANZADA DE MOVIMIENTO

- **Arco:** Hace que el robot se mueva en forma de curva desde donde está hasta un punto final, usando el sistema de coordenadas cartesianas. Para que funcione, los tres puntos (inicio, medio y final) no deben estar en línea recta.
- **Círculo:** Hace que el robot se mueva en un círculo desde su posición actual, dando el número de vueltas que indiques y luego regresando al punto de inicio. Los tres puntos que definen el círculo (inicio, medio y final) no deben estar en línea recta, y el círculo debe estar dentro del área que el brazo puede alcanzar.



CONFIGURACIÓN AVANZADA DE MOVIMIENTO

- **Trayectoria continua (CP):** Es un tipo de movimiento en el que el brazo del robot va desde un punto de inicio hasta un punto final, pasando por un punto intermedio. El robot puede pasar por ese punto haciendo una curva o girando en ángulo recto, sin detenerse.



BLOQUES DE POSTURA

Los bloques de postura en Dobot Studio Pro permiten al robot alcanzar posiciones específicas definidas por el usuario, facilitando tareas como la calibración, la repetición de movimientos y la programación de trayectorias complejas.

- **Obtener coordenadas cartesianas de la postura actual:** Consigue las coordenadas (posiciones en X, Y, Z) donde está ubicado el robot en este momento
- **Obtener valor** Toma el valor de una posición específica (como X, Y o Z) de la ubicación actual del robot.

Gets the value of the current Cartesian position

Gets the value of the current Cartesian position

BLOQUES DE POSTURA

- **Obtener coordenadas articulares de la postura actual:** Consigue los ángulos o posiciones de cada articulación del robot en su posición actual.
- **Obtener valor articular específico de la postura actual:** Obtiene el valor (ángulo o posición) de una articulación específica del robot en la postura actual.

Gets the value of the current joint position

Gets the value of the current joint position

BLOQUES DE POSTURA

- **Obtener coordenadas tras desplazamiento:** Consigue las coordenadas de un punto después de moverlo una cierta distancia o dirección.
- **Establecer coordenadas cartesianas:** Define una posición usando los valores X, Y y Z dentro del sistema de coordenadas cartesianas.

Cartesian position offset: Δx Δy Δz ΔR from position

Custom Cartesian point X Y Z R User Tool

BLOQUES DE POSTURA

- **Obtener coordenadas:** Saca las posiciones (valores de X, Y, Z) de un punto específico en el espacio usando el sistema cartesiano.
- **Obtener coordenadas de un eje específico:** Obtiene el valor solo de un eje (X, Y o Z) del punto que elijas en el sistema cartesiano.

Point

InitialPose ▼

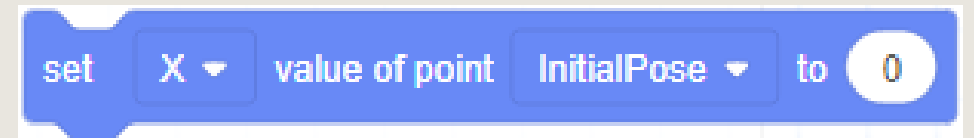
X ▼

value of point

InitialPose ▼

BLOQUES DE POSTURA

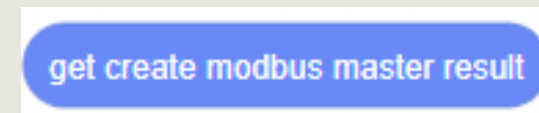
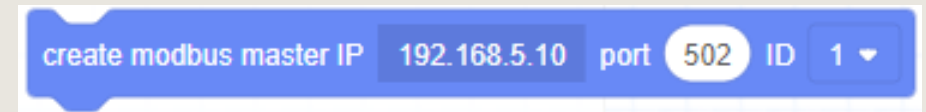
- **Ajustar coordenadas:** Cambiar el valor de una posición específica en uno de los ejes (X, Y o Z) dentro del sistema de coordenadas cartesianas.



BLOQUES DE MODBUS

En Dobot Studio Pro, los bloques de Modbus permiten integrar el robot con sistemas externos como PLCs, HMIs o SCADAs, facilitando la automatización industrial mediante el protocolo de comunicación Modbus.

- **Crear maestro Modbus:** Configura un maestro Modbus para conectarlo y comunicarse con un dispositivo esclavo.
- **Obtener resultado de creación del maestro Modbus:** Consulta si la configuración del maestro Modbus fue exitosa o si hubo algún problema.



BLOQUES DE MODBUS

- **Esperar registro de entrada:** El programa se detiene hasta que el valor en una dirección específica del registro de entrada cumpla una condición que tú defines. Luego continúa con el siguiente comando.
- **Esperar registro de retención:** El programa hace una pausa hasta que el valor en una dirección específica del registro de retención cumpla una condición, y después sigue con los siguientes pasos.

Waiting until input register address type is

Waiting until holding register address type is

BLOQUES DE MODBUS

- **Esperar registro de entrada discreta:** Detiene el programa hasta que el valor en una dirección específica del registro de entrada discreta cumpla la condición que pidas. Luego continúa con el siguiente paso.
- **Esperar registro de bobina:** Espera hasta que el valor en una dirección específica del registro de bobina cumpla la condición que definas, y después sigue con el siguiente comando.

Waiting until discrete input register address is

Waiting until coils register address is

BLOQUES DE MODBUS

- **Obtener registro de entrada:**
Consigue el valor que está guardado en una dirección específica del registro de entrada.
- **Obtener registro de retención:**
Obtiene el valor guardado en una dirección específica del registro de retención.

get input register address type U16 ▼

get holding register address type U16 ▼

BLOQUES DE MODBUS

- **Obtener entrada discreta:** Recupera el valor (encendido o apagado) que está en una dirección específica del registro de entrada discreta.
- **Obtener registro de bobina:** Obtiene el valor (encendido o apagado) almacenado en una dirección específica del registro de bobina.

get discrete input register address

0

get coils register address

0

BLOQUES DE MODBUS

- **Obtener entrada discreta:** Recupera el valor (encendido o apagado) de una entrada digital específica.
- **Obtener varios valores del registro de bobina:** Recupera varios valores (encendido o apagado) desde una dirección específica del registro de bobinas.
- **Obtener varios valores del registro de retención:** Recupera varios valores guardados desde una dirección específica del registro de retención.

get coils register array address 0 bits 1

set holding register address 0 data 50 type U16 ▼

setting multiple coil registers address 0 data 0,0,0,0,0,0,0,0,0,0

BLOQUES DE MODBUS

- **Configurar registro de bobina:**
Escribe un valor (como encendido o apagado) en una dirección específica del registro de bobina.
- **Configurar múltiples registros de bobina:** Escribe varios valores en diferentes direcciones del registro de bobina al mismo tiempo.

set coils register address 0 data 0 ▼

setting multiple coil registers address 0 data 0,0,0,0,0,0,0,0,0,0

BLOQUES DE MODBUS

- **Configurar registro de retención:** Escribe un valor en una dirección específica del registro de retención.
- **Cerrar maestro Modbus:** Termina la conexión del maestro Modbus y desconecta todos los dispositivos que están conectados como esclavos.

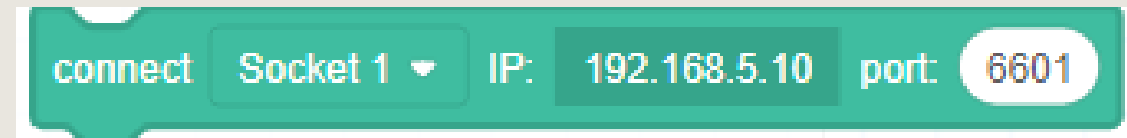
set holding register address data type

close modbus master

BLOQUES DE COMANDOS TCP

Permiten que el robot se comuniquen con dispositivos externos a través del protocolo TCP/IP, facilitando la integración con sistemas como PLCs, HMIs o cámaras inteligentes.

- **Establecer conexión de socket:** Crea una conexión para comunicarse con otro servidor usando TCP.
- **Obtener resultado de conexión de socket:** Verifica si la conexión TCP se hizo correctamente o si hubo algún problema.



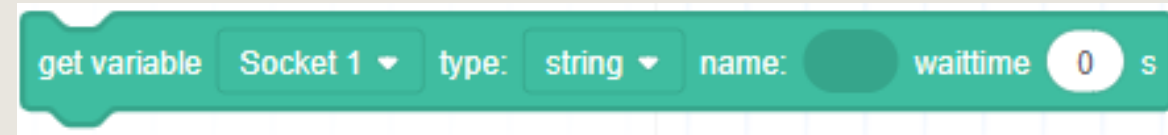
BLOQUES DE COMANDOS TCP

- **Crear socket:** Configura un servidor TCP que espera a que un cliente se conecte.
- **Obtener resultado de creación de socket:** Verifica si el servidor TCP se creó correctamente o si hubo algún problema.



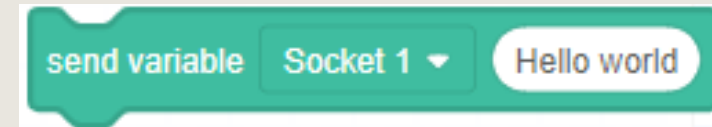
BLOQUES DE COMANDOS TCP

- **Cerrar socket:** Termina la conexión del socket que está abierta y corta la comunicación.
- **Recuperar variables:** Obtiene datos o variables a través de la comunicación TCP y las guarda para usarlas.



BLOQUES DE COMANDOS TCP

- **Enviar variables:** Envía datos o variables a través de la comunicación TCP hacia otro dispositivo.
- **Obtener resultado del envío de variables:** Verifica si el envío de las variables fue exitoso o si hubo algún error.



LUA BASIC GRAMMAR (PYTHON)

MOVIMIENTO

- **MovJ:** Mueve el robot desde donde está hasta un punto destino usando un movimiento que junta las articulaciones para llegar rápido, pero sin seguir una línea recta exacta.
- **MovL:** Mueve el robot en línea recta desde su posición actual hasta el punto destino, ideal cuando necesitas que el extremo del brazo siga un camino recto.

MovJ(P, {CP=1, SpeedJ=50, AccJ=20, SYNC=0})

MovL(P, {CP=1, SpeedJ=50, AccJ=20, SYNC=0})

MOVIMIENTO

- **Jump:** El robot se mueve rápido desde su posición actual hasta un punto objetivo, generalmente levantándose primero para evitar obstáculos. No crea un servidor TCP
- .
- **ConjuntoMovJ:** El robot mueve sus articulaciones (como las "rodillas" o "codos") desde su posición actual hasta unos ángulos específicos, usando un movimiento conjunto.

```
Jump(P, {SpeedL=50, AccL=20, Start=10, ZLimit=100, End=20, SYNC=0})  
Jump(P, {SpeedL=50, AccL=20, Arch=1, SYNC=0})
```

```
JointMovJ(P, {CP=1, SpeedL=50, AccJ=20, SYNC=0})
```

MOVIMIENTO

- **Círculo:** El robot se mueve desde donde está en una trayectoria circular y regresa al punto inicial después de dar el número de vueltas que le indiqués.
- **Arco:** El robot se mueve desde su posición actual hasta un punto destino siguiendo una curva en forma de arco dentro del sistema de coordenadas cartesianas.

Circle(P1,P2,Count,{CP=1, SpeedL=50, AccL=20, SYNC=0})

Arc3(P1,P2, {CP=1, SpeedL=50, AccL=20, SYNC=0})

MOVIMIENTO

- **MovJIO:** El robot se mueve desde su posición actual hasta un punto destino usando un movimiento articular (que mueve sus "articulaciones" punto por punto). Durante ese movimiento, puede encender o apagar un puerto de salida digital (como activar una luz o un motor).

MovJIO(P, {Mode, Distance, Index, Status},{Mode, Distance, Index, Status}...},
{CP=1, SpeedJ=50, AccJ=20, SYNC=0})

- **MovLIO:** robot se mueve en línea recta desde donde está hasta un punto destino, y mientras se mueve puede cambiar el estado de un puerto de salida digital (por ejemplo, encender o apagar algo).

MovLIO(P, {Mode, Distance, Index, Status},{Mode, Distance, Index, Status}...},
{CP=1, SpeedL=50, AccL=20, SYNC=0})

MOVIMIENTO

- **MovJExt:** Mueve una parte extra del robot llamada articulación auxiliar hacia el ángulo y posición que quieras.

MovJExt(AD, {SpeddE=50, AccE=20, SYNC=0})

PARÁMETRO DE MOVIMIENTO

- **Sincronizar:** Este comando pausa el programa hasta que se terminen de ejecutar todos los comandos anteriores, evitando que se ejecuten comandos nuevos antes de tiempo.
- **CP:** Ajusta cómo se mueve el brazo del robot cuando pasa por varios puntos seguidos, permitiendo que lo haga de forma suave, ya sea girando en ángulo recto o en curvas sin detenerse.

Sync()

CP(R)

PARÁMETRO DE MOVIMIENTO

- **VelocidadJ:** Ajusta qué tan rápido se mueven las articulaciones del robot. La velocidad real depende del porcentaje que pongas, la velocidad configurada en la reproducción y la velocidad global.
- **AccJ:** Ajusta qué tan rápido aceleran las articulaciones del robot. La aceleración real depende del porcentaje que pongas, la aceleración configurada en la reproducción y la velocidad global.

SpeedJ(R)

AccJ(R)

PARÁMETRO DE MOVIMIENTO

- **VelocidadL:** Ajusta qué tan rápido se mueve el robot cuando hace movimientos en línea recta o en arco. La velocidad final depende del porcentaje que pongas, la configuración general y la velocidad global.
- **AccL:** Controla qué tan rápido acelera el robot en movimientos en línea recta o en arco. La aceleración real depende del porcentaje que pongas, la configuración general y la velocidad global.

SpeedL(R)

AccL(R)

PARÁMETRO DE MOVIMIENTO

- **ObtenerPose:** Consigue en tiempo real la posición actual del brazo del robot usando las coordenadas X, Y, Z (sistema cartesiano). Si usas un sistema de coordenadas especial o de herramienta, la posición será según ese sistema.
- **Obtener ángulo:** Obtiene en tiempo real el ángulo de las partes (articulaciones) del brazo del robot, según el sistema de coordenadas articulares.

GetPose()

GetAngle()

MOVIMIENTO RELATIVO

- **RelJoint:** Mueve los ejes J1 a J4 (las articulaciones del robot) desde una posición específica, sumando un ángulo para cada uno. Devuelve una nueva posición con esos cambios.
- **RelPoint:** Mueve un punto en el espacio cambiando sus valores en X, Y, Z y R (rotación) desde una posición dada. Devuelve una nueva posición con esos desplazamientos en el sistema cartesiano.

$\text{RelJoint}(P, \{\text{Offset1}, \text{Offset2}, \text{Offset3}, \text{Offset4}\})$

$\text{RelPoint}(P, \{\text{OffsetX}, \text{OffsetY}, \text{OffsetZ}, \text{OffsetR}\})$

MOVIMIENTO RELATIVO

- **RelMovJ:** Mueve el robot desde donde está hasta una nueva posición cercana usando un movimiento que junta las articulaciones (movimiento conjunto). La posición nueva se calcula como un desplazamiento relativo desde la posición actual.
 $\text{RelMovL}(\{\text{OffsetX}, \text{OffsetY}, \text{OffsetZ}, \text{OffsetR}\}, \{\text{CP}=1, \text{SpeedL}=50, \text{AccL}=20, \text{SYNC}=0\})$
- **RelMovL:** Mueve el robot desde donde está hasta una nueva posición cercana en línea recta. La posición nueva también se calcula como un desplazamiento relativo desde la posición actual.
 $\text{RelMovL}(\{\text{OffsetX}, \text{OffsetY}, \text{OffsetZ}, \text{OffsetR}\}, \{\text{CP}=1, \text{SpeedL}=50, \text{AccL}=20, \text{SYNC}=0\})$

IO

- **DI:** Obtiene el estado actual (encendido o apagado) de un puerto de entrada digital.

-

DO: Configura el estado (enciende o apaga) de un puerto de salida digital.

DI(index)

DO(index,ON|OFF)

IO

- **DOInstantáneo:** Mueve el robot desde su posición actual hasta una nueva posición cercana usando un movimiento conjunto punto a punto en coordenadas cartesianas.
- **HerramientaDI:** Obtiene el estado (encendido o apagado) del puerto de entrada digital que está conectado a la herramienta del robot.

DOInstant(index,ON|OFF)

ToolDI(index)

TCP/UDP

- **TCPCrear:** Configura una conexión de red TCP. Solo puede haber una red TCP activa a la vez.
- **TCPInicio:** Inicia la conexión TCP. Si el robot es servidor, espera que un cliente se conecte. Si es cliente, se conecta a un servidor.

TCPCreate(isServer, IP, port)

TCPStart(socket, timeout)

TCP/UDP

- **TCPLeer:** Recibe datos desde la conexión TCP que está activa.
- **TCPEscribir:** Envía datos a través de la conexión TCP hacia el dispositivo conectado.
- **TCPDestruir:** Cierra la conexión TCP y elimina el socket usado para comunicarse.

TCPRead(socket, timeout, **type**)

TCPWrite(socket, buf, timeout)

TCPDestroy(socket)

TCP/UDP

- **UDPCrear:** Configura una conexión de red UDP. Solo puedes tener una red UDP activa al mismo tiempo.
- **UDPLEer:** Recibe datos desde la conexión UDP establecida con otro dispositivo.
- **UDPEscribir:** Envía datos a través de la conexión UDP al dispositivo conectado.

UDPCreate(isServer, IP, port)

UDPRead(socket, timeout, **type**)

UDPWrite(socket, buf, timeout)

MODBUS

- **ModbusCrear:** Configura una estación maestra Modbus y conecta con la estación esclava para poder comunicarse.
- **ObtenerInBits:** Lee el estado (encendido o apagado) de las entradas digitales del esclavo Modbus.
- **ObtenerInRegs:** Lee los valores de registros de entrada del esclavo Modbus, usando el tipo de dato que especifiques.

ModbusCreate()

GetInBits(id, addr, count)

GetInRegs(id, addr, count, type)

MODBUS

- **ObtenerBobinas:** Lee los valores (encendido o apagado) de los registros de bobina del dispositivo Modbus esclavo.
- **ObtenerRegistrosDeRetención:** Lee los valores almacenados en los registros de retención del esclavo Modbus, usando el tipo de dato que indique.

GetCoils(id, addr, count)

GetHoldRegs(id, addr, count, type)

MODBUS

- **Conjunto de bobinas:** Escribe un valor (como encendido o apagado) en una dirección específica del registro de bobina.
- **Establecer registros de retención:** Escribe un valor específico en una dirección del registro de retención, usando el tipo de dato que elijas.
- **ModbusCerrar:** Cierra la conexión con el dispositivo esclavo Modbus.

SetCoils(id, addr, count, table)

SetHoldRegs(id, count, table, type)

ModbusClose(id)

CONTROL DE PROGRAMA

- **Dormir:** Hace que el programa espere un tiempo específico antes de seguir con la siguiente instrucción.
- **Esperar:** Pausa la ejecución hasta que un movimiento termine o retrasa el siguiente comando por un tiempo.
- **Pausa:** Detiene completamente el programa hasta que alguien lo reanude manualmente o de forma remota.

Sleep(time)

Wait(time)

Pause()

CONTROL DE PROGRAMA

- **Establecer nivel de colisión:** Configura el límite para que el robot detecte si ha chocado con algo.
- **Restablecer tiempo transcurrido:** Reinicia un temporizador para medir cuánto tiempo tarda una acción. Se usa junto con el comando que mide el tiempo transcurrido.

SetCollisionLevel(level)

ElapsedTime()

CONTROL DE PROGRAMA

- **Tiempo del sistema:** Obtiene la hora actual del sistema o reloj del robot.
- **Establecer carga útil:** Ajusta el peso y la posición de la carga que lleva el robot, para que pueda moverse con seguridad.
- **SetTool485:** Configura cómo se envían y reciben los datos por la conexión RS485 de la herramienta que está en el extremo del robot.

Systeme()

SetPayload(payload, {x, y}, index)

SetTool485(baud,parity,stopbit)

CONTROL DE PROGRAMA

- **Establecer usuario:** Cambia el sistema de coordenadas (el punto de referencia) que usa un usuario específico para mover el robot.
- **Establecer herramienta:** Cambia el sistema de coordenadas que usa una herramienta específica conectada al robot para trabajar.

SetUser(index,table)

SetTool(index,table)